Object-Oriented Programming Through Java Lab

Programs, Source Code and Outputs

II Year B.Tech. AI & ML - I Semester | Course code 23A05303P | JNTUA R23

All console programs were compiled and run with OpenJDK 21. The outputs shown are the actual results of those runs, and sample inputs are shown in the output label. Programs that use threads print in timing order, so lines printed at the same instant may appear in a different order between runs.

JavaFX GUI programs (Exercise 8 items 9 and 10) and JDBC programs (Exercise 9) need extra software. Each one has a note explaining how to run it and what to expect.

Contents

1. Exercise 1(a): Default values of primitive data types
2. Exercise 1(b): Roots of a quadratic equation
3. Exercise 2(a): Binary search
4. Exercise 2(b): Bubble sort
5. Exercise 2(c): StringBuffer - delete and remove characters
6. Exercise 3(a): Class, methods and invoking them from main
7. Exercise 3(b): Method overloading
8. Exercise 3(c): Constructor
9. Exercise 3(d): Constructor overloading
10. Exercise 4(a): Single inheritance
11. Exercise 4(b): Multilevel inheritance
12. Exercise 4(c): Abstract class to find areas of shapes
13. Exercise 5(a): The super keyword
14. Exercise 5(b): Interface and the kind of inheritance achieved
15. Exercise 5(c): Runtime polymorphism
16. Exercise 6(a): Exception handling mechanism
17. Exercise 6(b): Multiple catch clauses
18. Exercise 6(c): Built-in exceptions
19. Exercise 6(d): User-defined exception
20. Exercise 7(a): Threads by extending the Thread class
21. Exercise 7(a) (repeated): Threads by implementing Runnable
22. Exercise 7(b): isAlive() and join()
23. Exercise 7(c): Daemon threads
24. Exercise 7(d): Producer-Consumer problem
25. Exercise 8: Importing and using user-defined packages
26. Exercise 9 (item 4): Connect to a database using JDBC
27. Exercise 9 (item b): Insert values into a database using JDBC
28. Exercise 9 (item c): Delete values from a database using JDBC
29. Exercise 8 (item 9): GUI with a label and an ImageView (JavaFX)
30. Exercise 8 (item 10): Tip Calculator app (JavaFX)

Exercise 1(a): Default values of primitive data types

Question: Write a Java program to display the default value of all primitive data types of Java.

Source code: DefaultValues.java

```
public class DefaultValues {
    // Static fields get default values automatically, just like instance fields.
    // Local variables do NOT get default values and must be initialised before use.
    static byte b;
    static short s;
    static int i;
    static long l;
    static float f;
    static double d;
    static char c;
    static boolean flag;

    public static void main(String[] args) {
        System.out.println("Default values of primitive data types in Java:");
        System.out.println("byte    : " + b);
        System.out.println("short   : " + s);
        System.out.println("int     : " + i);
        System.out.println("long    : " + l);
        System.out.println("float   : " + f);
        System.out.println("double  : " + d);
        System.out.println("char    : code " + (int) c + " (the NUL character)");
        System.out.println("boolean : " + flag);
    }
}
```

Output

```
Default values of primitive data types in Java:
byte    : 0
short   : 0
int     : 0
long    : 0
float   : 0.0
double  : 0.0
char    : code 0 (the NUL character)
boolean : false
```

Exercise 1(b): Roots of a quadratic equation

Question: Write a Java program that displays the roots of a quadratic equation ax^2+bx+c=0. Calculate the discriminant D and, based on its value, describe the nature of the roots.

Source code: QuadraticRoots.java

```
import java.util.Scanner;

public class QuadraticRoots {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a (coefficient of x^2): ");
        double a = sc.nextDouble();
        System.out.print("Enter b (coefficient of x): ");
        double b = sc.nextDouble();
        System.out.print("Enter c (constant term): ");
        double c = sc.nextDouble();

        if (a == 0) {
            System.out.println("Not a quadratic equation because a must not be 0.");
            sc.close();
            return;
        }

        double d = b * b - 4 * a * c;   // discriminant
        System.out.println("Equation: " + a + "x^2 + " + b + "x + " + c + " = 0");
        System.out.println("Discriminant D = " + d);

        if (d > 0) {
            double r1 = (-b + Math.sqrt(d)) / (2 * a);
            double r2 = (-b - Math.sqrt(d)) / (2 * a);
            System.out.println("D > 0: the roots are real and distinct.");
            System.out.println("Root 1 = " + r1);
            System.out.println("Root 2 = " + r2);
        } else if (d == 0) {
            double r = -b / (2 * a);
            System.out.println("D = 0: the roots are real and equal.");
            System.out.println("Root 1 = Root 2 = " + r);
        } else {
            double real = -b / (2 * a);
            double imag = Math.abs(Math.sqrt(-d) / (2 * a));
            System.out.println("D < 0: the roots are complex (imaginary) and conjugate.");
            System.out.println("Root 1 = " + real + " + " + imag + "i");
            System.out.println("Root 2 = " + real + " - " + imag + "i");
        }
        sc.close();
    }
}
```

Output - D > 0: a=1, b=-3, c=2 (input: 1 -3 2)

```
Enter a (coefficient of x^2): Enter b (coefficient of x): Enter c (constant term): Equation:
 1.0x^2 + -3.0x + 2.0 = 0
Discriminant D = 1.0
D > 0: the roots are real and distinct.
Root 1 = 2.0
Root 2 = 1.0
```

Output - D = 0: a=1, b=2, c=1 (input: 1 2 1)

```
Enter a (coefficient of x^2): Enter b (coefficient of x): Enter c (constant term): Equation:
 1.0x^2 + 2.0x + 1.0 = 0
Discriminant D = 0.0
D = 0: the roots are real and equal.
Root 1 = Root 2 = -1.0
```

Output - D < 0: a=1, b=1, c=1 (input: 1 1 1)

```
Enter a (coefficient of x^2): Enter b (coefficient of x): Enter c (constant term): Equation:
 1.0x^2 + 1.0x + 1.0 = 0
Discriminant D = -3.0
D < 0: the roots are complex (imaginary) and conjugate.
Root 1 = -0.5 + 0.8660254037844386i
Root 2 = -0.5 - 0.8660254037844386i
```

Exercise 2(a): Binary search

Question: Write a Java program to search for an element in a given list of elements using the binary search mechanism.

Source code: BinarySearch.java

```
import java.util.Arrays;
import java.util.Scanner;

public class BinarySearch {

    // Iterative binary search: returns the index of key, or -1 if it is not present
    static int binarySearch(int[] arr, int key) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = (low + high) >>> 1;
            if (arr[mid] == key) {
                return mid;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] list = new int[n];
        System.out.print("Enter " + n + " elements: ");
        for (int i = 0; i < n; i++) {
            list[i] = sc.nextInt();
        }

        // Binary search needs sorted data, so sort the list first
        Arrays.sort(list);
        System.out.println("Sorted list: " + Arrays.toString(list));

        System.out.print("Enter element to search: ");
        int key = sc.nextInt();
        int index = binarySearch(list, key);
        if (index != -1) {
            System.out.println("Element " + key + " found at index " + index);
        } else {
            System.out.println("Element " + key + " not found in the list");
        }
        sc.close();
    }
}
```

Output - Element present (input: 7 10 25 3 47 16 8 30 25)

```
Enter number of elements: Enter 7 elements: Sorted list: [3, 8, 10, 16, 25, 30, 47]
Enter element to search: Element 25 found at index 4
```

Output - Element absent (input: 7 10 25 3 47 16 8 30 99)

```
Enter number of elements: Enter 7 elements: Sorted list: [3, 8, 10, 16, 25, 30, 47]
Enter element to search: Element 99 not found in the list
```

Exercise 2(b): Bubble sort

Question: Write a Java program to sort a given list of elements using bubble sort, then find an element in the sorted list.

Source code: BubbleSortSearch.java

```
import java.util.Arrays;
import java.util.Scanner;

public class BubbleSortSearch {

    static void bubbleSort(int[] a) {
        int n = a.length;
        for (int pass = 0; pass < n - 1; pass++) {
            boolean swapped = false;
            for (int j = 0; j < n - 1 - pass; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                    swapped = true;
                }
            }
            System.out.println("After pass " + (pass + 1) + ": " + Arrays.toString(a));
            if (!swapped) {
                break;   // list is already sorted
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] list = new int[n];
        System.out.print("Enter " + n + " elements: ");
        for (int i = 0; i < n; i++) {
            list[i] = sc.nextInt();
        }

        System.out.println("\nBubble sort passes:");
        bubbleSort(list);
        System.out.println("Final sorted list: " + Arrays.toString(list));

        System.out.print("\nEnter element to search: ");
        int key = sc.nextInt();
        int position = -1;
        for (int i = 0; i < n; i++) {
            if (list[i] == key) {
                position = i + 1;   // 1-based position
                break;
            }
        }
        if (position != -1) {
            System.out.println("Element " + key + " is at position " + position + " in the sorted list");
        } else {
            System.out.println("Element " + key + " is not in the list");
        }
        sc.close();
    }
}
```

Output - sample input (input: 5 64 34 25 12 22 22)

```
Enter number of elements: Enter 5 elements:
Bubble sort passes:
After pass 1: [34, 25, 12, 22, 64]
After pass 2: [25, 12, 22, 34, 64]
After pass 3: [12, 22, 25, 34, 64]
After pass 4: [12, 22, 25, 34, 64]
Final sorted list: [12, 22, 25, 34, 64]

Enter element to search: Element 22 is at position 2 in the sorted list
```

Exercise 2(c): StringBuffer - delete and remove characters

Question: Write a Java program using StringBuffer to delete and remove characters.

Source code: StringBufferDemo.java

```
public class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Java Programming");
        System.out.println("Original text          : " + sb);

        sb.delete(0, 5);                      // remove characters from index 0 up to (not incl.) 5
        System.out.println("After delete(0, 5)     : " + sb);

        sb.deleteCharAt(0);                   // remove the single character at index 0
        System.out.println("After deleteCharAt(0)  : " + sb);

        // Remove every occurrence of a given character
        char target = 'm';
        int i = 0;
        while (i < sb.length()) {
            if (sb.charAt(i) == target) {
                sb.deleteCharAt(i);
            } else {
                i++;
            }
        }
        System.out.println("After removing 'm'     : " + sb);

        sb.reverse();
        System.out.println("Reversed               : " + sb);
        System.out.println("Length of the buffer   : " + sb.length());
    }
}
```

Output

```
Original text          : Java Programming
After delete(0, 5)     : Programming
After deleteCharAt(0)  : rogramming
After removing 'm'     : rograing
Reversed               : gniargor
Length of the buffer   : 8
```

Exercise 3(a): Class, methods and invoking them from main

Question: Write a Java program to implement the class mechanism. Create a class, its methods, and invoke them inside the main method.

Source code: ClassDemo.java

```
class Student {
    private String name;
    private int rollNo;
    private int[] marks = new int[3];

    void readData(String n, int r, int m1, int m2, int m3) {
        name = n;
        rollNo = r;
        marks[0] = m1;
        marks[1] = m2;
        marks[2] = m3;
    }

    int total() {
        return marks[0] + marks[1] + marks[2];
    }

    double average() {
        return total() / 3.0;
    }

    void display() {
        System.out.println("Roll No : " + rollNo);
        System.out.println("Name    : " + name);
        System.out.println("Total   : " + total());
        System.out.printf("Average : %.2f%n", average());
    }
}

public class ClassDemo {
    public static void main(String[] args) {
        Student s = new Student();              // create an object of the class
        s.readData("Ravi", 101, 78, 85, 90);    // invoke methods from main
        s.display();
    }
}
```

Output

```
Roll No : 101
Name    : Ravi
Total   : 253
Average : 84.33
```

Exercise 3(b): Method overloading

Question: Write a Java program to implement method overloading.

Source code: MethodOverloadDemo.java

```
public class MethodOverloadDemo {
    static int add(int a, int b) { return a + b; }
    static double add(double a, double b) { return a + b; }
    static int add(int a, int b, int c) { return a + b + c; }
    static String add(String a, String b) { return a + b; }

    static void show(int x) { System.out.println("show(int) called with " + x); }
    static void show(String s) { System.out.println("show(String) called with " + s); }

    public static void main(String[] args) {
        System.out.println("add(2, 3)          = " + add(2, 3));
        System.out.println("add(2.5, 3.5)      = " + add(2.5, 3.5));
        System.out.println("add(1, 2, 3)       = " + add(1, 2, 3));
        System.out.println("add(\"Java\", \"Lab\") = " + add("Java", "Lab"));
        show(10);
        show("Hello");
    }
}
```

Output

```
add(2, 3)          = 5
add(2.5, 3.5)      = 6.0
add(1, 2, 3)       = 6
add("Java", "Lab") = JavaLab
show(int) called with 10
show(String) called with Hello
```

Exercise 3(c): Constructor

Question: Write a Java program to implement a constructor.

Source code: ConstructorDemo.java

```
class Box {
    double width;
    double height;

    // Default constructor
    Box() {
        width = 1;
        height = 1;
        System.out.println("Default constructor called");
    }

    // Parameterized constructor (this refers to the current object's fields)
    Box(double width, double height) {
        this.width = width;
        this.height = height;
        System.out.println("Parameterized constructor called");
    }

    double area() {
        return width * height;
    }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        Box b1 = new Box();
        System.out.println("Area of b1 = " + b1.area());

        Box b2 = new Box(4, 5);
        System.out.println("Area of b2 = " + b2.area());
    }
}
```

Output

```
Default constructor called
Area of b1 = 1.0
Parameterized constructor called
Area of b2 = 20.0
```

Exercise 3(d): Constructor overloading

Question: Write a Java program to implement constructor overloading.

Source code: ConstructorOverloadDemo.java

```
class Rectangle {
    double length;
    double breadth;

    Rectangle() {
        this(1, 1);                      // calls the two-argument constructor
        System.out.println("No-argument constructor finished");
    }

    Rectangle(double side) {
        this(side, side);                // square: calls the two-argument constructor
        System.out.println("One-argument (square) constructor finished");
    }

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
        System.out.println("Two-argument constructor called");
    }

    double area() {
        return length * breadth;
    }
}

public class ConstructorOverloadDemo {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        System.out.println("Area r1 = " + r1.area() + "\n");

        Rectangle r2 = new Rectangle(5);
        System.out.println("Area r2 = " + r2.area() + "\n");

        Rectangle r3 = new Rectangle(6, 3);
        System.out.println("Area r3 = " + r3.area());
    }
}
```

Output

```
Two-argument constructor called
No-argument constructor finished
Area r1 = 1.0

Two-argument constructor called
One-argument (square) constructor finished
Area r2 = 25.0

Two-argument constructor called
Area r3 = 18.0
```

Exercise 4(a): Single inheritance

Question: Write a Java program to implement single inheritance.

Source code: SingleInheritanceDemo.java

```
class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    void eat() {
        System.out.println(name + " is eating");
    }
}

// Single inheritance: Dog inherits from one parent class (Animal)
class Dog extends Animal {
    Dog(String name) {
        super(name);
    }

    void bark() {
        System.out.println(name + " says: Woof!");
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog("Tommy");
        d.eat();     // inherited from Animal
        d.bark();    // defined in Dog
    }
}
```

Output

```
Tommy is eating
Tommy says: Woof!
```

Exercise 4(b): Multilevel inheritance

Question: Write a Java program to implement multilevel inheritance.

Source code: MultilevelInheritanceDemo.java

```
class Animal {
    void eat() {
        System.out.println("Animals eat food");
    }
}

class Mammal extends Animal {
    void breathe() {
        System.out.println("Mammals breathe air with lungs");
    }
}

// Multilevel inheritance: Dog -> Mammal -> Animal
class Dog extends Mammal {
    void bark() {
        System.out.println("Dog barks: Woof!");
    }
}

public class MultilevelInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();      // from Animal (grandparent)
        d.breathe();  // from Mammal (parent)
        d.bark();     // from Dog
    }
}
```

Output

```
Animals eat food
Mammals breathe air with lungs
Dog barks: Woof!
```

Exercise 4(c): Abstract class to find areas of shapes

Question: Write a Java program for an abstract class to find the areas of different shapes.

Source code: AbstractShapeDemo.java

```
abstract class Shape {
    abstract double area();
    abstract double perimeter();

    void printInfo() {
        System.out.printf("%-10s area = %8.2f   perimeter = %8.2f%n",
                getClass().getSimpleName(), area(), perimeter());
    }
}

class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
    double perimeter() { return 2 * Math.PI * r; }
}

class Rectangle extends Shape {
    double w, h;
    Rectangle(double w, double h) { this.w = w; this.h = h; }
    double area() { return w * h; }
    double perimeter() { return 2 * (w + h); }
}

class Triangle extends Shape {
    double a, b, c;
    Triangle(double a, double b, double c) { this.a = a; this.b = b; this.c = c; }
    double area() {
        // Heron's formula
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }
    double perimeter() { return a + b + c; }
}

public class AbstractShapeDemo {
    public static void main(String[] args) {
        // Shape s = new Shape();   // compile error: an abstract class cannot be instantiated
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6),
            new Triangle(3, 4, 5)
        };
        for (Shape s : shapes) {
            s.printInfo();
        }
    }
}
```

Output

```
Circle     area =    78.54   perimeter =    31.42
Rectangle  area =    24.00   perimeter =    20.00
Triangle   area =     6.00   perimeter =    12.00
```

Exercise 5(a): The super keyword

Question: Write a Java program giving an example of the super keyword.

Source code: SuperKeywordDemo.java

```
class Parent {
    String name = "Parent";

    Parent() {
        System.out.println("Parent constructor called");
    }

    void show() {
        System.out.println("show() in Parent");
    }
}

class Child extends Parent {
    String name = "Child";

    Child() {
        super();                  // calls the Parent constructor (optional here, shown for clarity)
        System.out.println("Child constructor called");
    }

    @Override
    void show() {
        super.show();             // calls the parent version of show()
        System.out.println("show() in Child");
    }

    void display() {
        System.out.println("this.name  = " + this.name);
        System.out.println("super.name = " + super.name);
    }
}

public class SuperKeywordDemo {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();
        c.display();
    }
}
```

Output

```
Parent constructor called
Child constructor called
show() in Parent
show() in Child
this.name  = Child
super.name = Parent
```

Exercise 5(b): Interface and the kind of inheritance achieved

Question: Write a Java program to implement an interface. What kind of inheritance can be achieved?

Source code: InterfaceDemo.java

```
interface Printable {
    void print();
}

interface Scannable {
    void scan();
}

// One class implements two interfaces: multiple inheritance of type
class MultiFunctionPrinter implements Printable, Scannable {
    @Override
    public void print() {
        System.out.println("Printing a document...");
    }

    @Override
    public void scan() {
        System.out.println("Scanning a document...");
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        MultiFunctionPrinter mfp = new MultiFunctionPrinter();
        mfp.print();
        mfp.scan();

        Printable p = mfp;   // the same object viewed through each interface type
        Scannable s = mfp;
        p.print();
        s.scan();

        System.out.println("\nInheritance achieved through interfaces: multiple inheritance of type");
        System.out.println("(a class cannot extend two classes, but it can");
        System.out.println(" implement many interfaces).");
    }
}
```

Output

```
Printing a document...
Scanning a document...
Printing a document...
Scanning a document...

Inheritance achieved through interfaces: multiple inheritance of type
(a class cannot extend two classes, but it can
 implement many interfaces).
```

Exercise 5(c): Runtime polymorphism

Question: Write a Java program that implements runtime polymorphism.

Source code: RuntimePolymorphismDemo.java

```
class Shape {
    void draw() {
        System.out.println("Drawing a generic shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a circle");
    }
}

class Square extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a square");
    }
}

public class RuntimePolymorphismDemo {
    public static void main(String[] args) {
        Shape[] shapes = { new Shape(), new Circle(), new Square() };

        // The method is chosen at run time based on the actual object type
        for (Shape s : shapes) {
            s.draw();
        }
    }
}
```

Output

```
Drawing a generic shape
Drawing a circle
Drawing a square
```

Exercise 6(a): Exception handling mechanism

Question: Write a Java program that describes the exception handling mechanism.

Source code: ExceptionDemo.java

```
public class ExceptionDemo {

    static int divide(int a, int b) {
        return a / b;
    }

    public static void main(String[] args) {
        try {
            System.out.println("10 / 2 = " + divide(10, 2));
            System.out.println("10 / 0 = " + divide(10, 0));   // throws ArithmeticException
            System.out.println("This line is never reached");
        } catch (ArithmeticException e) {
            System.out.println("Caught exception: " + e);
        } finally {
            System.out.println("finally block always runs");
        }
        System.out.println("Program continues after exception handling");
    }
}
```

Output

```
10 / 2 = 5
Caught exception: java.lang.ArithmeticException: / by zero
finally block always runs
Program continues after exception handling
```

Exercise 6(b): Multiple catch clauses

Question: Write a Java program illustrating multiple catch clauses.

Source code: MultipleCatchDemo.java

```
public class MultipleCatchDemo {

    static void test(int[] data, String text, int divisor, int index) {
        try {
            int value = data[index] / divisor;
            int parsed = Integer.parseInt(text);
            System.out.println("Result = " + (value + parsed));
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException: " + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("NumberFormatException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Some other exception: " + e);
        }
    }

    public static void main(String[] args) {
        int[] data = {10, 20, 30};
        System.out.println("Case 1 (all valid):");
        test(data, "5", 2, 1);
        System.out.println("Case 2 (division by zero):");
        test(data, "5", 0, 1);
        System.out.println("Case 3 (bad index):");
        test(data, "5", 2, 10);
        System.out.println("Case 4 (bad number text):");
        test(data, "abc", 2, 1);
    }
}
```

Output

```
Case 1 (all valid):
Result = 15
Case 2 (division by zero):
ArithmeticException: / by zero
Case 3 (bad index):
ArrayIndexOutOfBoundsException: Index 10 out of bounds for length 3
Case 4 (bad number text):
NumberFormatException: For input string: "abc"
```

Exercise 6(c): Built-in exceptions

Question: Write a Java program for the creation of Java built-in exceptions.

Source code: BuiltInExceptionsDemo.java

```
public class BuiltInExceptionsDemo {
    public static void main(String[] args) {
        try {
            int[] a = new int[3];
            a[5] = 10;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("1. " + e);
        }

        try {
            String s = null;
            System.out.println(s.length());
        } catch (NullPointerException e) {
            System.out.println("2. NullPointerException caught");
        }

        try {
            int n = Integer.parseInt("12a");
            System.out.println(n);
        } catch (NumberFormatException e) {
            System.out.println("3. " + e);
        }

        try {
            int x = 5 / 0;
            System.out.println(x);
        } catch (ArithmeticException e) {
            System.out.println("4. " + e);
        }

        try {
            Object o = "hello";
            Integer i = (Integer) o;
            System.out.println(i);
        } catch (ClassCastException e) {
            System.out.println("5. ClassCastException caught");
        }

        try {
            System.out.println("abc".charAt(10));
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("6. " + e);
        }

        try {
            int[] negative = new int[-2];
            System.out.println(negative.length);
        } catch (NegativeArraySizeException e) {
            System.out.println("7. " + e);
        }
    }
}
```

Output

```
1. java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
2. NullPointerException caught
3. java.lang.NumberFormatException: For input string: "12a"
4. java.lang.ArithmeticException: / by zero
5. ClassCastException caught
6. java.lang.StringIndexOutOfBoundsException: Index 10 out of bounds for length 3
7. java.lang.NegativeArraySizeException: -2
```

Exercise 6(d): User-defined exception

Question: Write a Java program for the creation of a user-defined exception.

Source code: UserDefinedExceptionDemo.java

```
class InsufficientBalanceException extends Exception {
    private final double shortBy;

    InsufficientBalanceException(String message, double shortBy) {
        super(message);
        this.shortBy = shortBy;
    }

    double getShortBy() {
        return shortBy;
    }
}

class BankAccount {
    private double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException(
                    "Withdrawal of " + amount + " exceeds balance of " + balance,
                    amount - balance);
        }
        balance -= amount;
        System.out.println("Withdrawn " + amount + ". Remaining balance: " + balance);
    }
}

public class UserDefinedExceptionDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(1000);
        try {
            account.withdraw(500);
            account.withdraw(800);      // should fail
        } catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("You are short by " + e.getShortBy());
        }
    }
}
```

Output

```
Withdrawn 500.0. Remaining balance: 500.0
Error: Withdrawal of 800.0 exceeds balance of 500.0
You are short by 300.0
```

Exercise 7(a): Threads by extending the Thread class

Question: Write a Java program that creates threads by extending the Thread class. The first thread displays "Good Morning" every 1 second, the second displays "Hello" every 2 seconds, and the third displays "Welcome" every 3 seconds.

Source code: ThreadDemo.java

```
class MorningThread extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println("Good Morning");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

class HelloThread extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println("Hello");
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

class WelcomeThread extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println("Welcome");
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        new MorningThread().start();
        new HelloThread().start();
        new WelcomeThread().start();
    }
}
```

Output

```
Good Morning
Hello
Welcome
Good Morning
Hello
Good Morning
Welcome
Hello
Welcome
```

Note: *Threads run concurrently, so lines that fall at the same moment (for example at t = 0 s) can appear in a different order from run to run.*

Exercise 7(a) (repeated): Threads by implementing Runnable

Question: Repeat the same program by implementing the Runnable interface.

Source code: RunnableDemo.java

```
class MorningTask implements Runnable {
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println("Good Morning");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

class HelloTask implements Runnable {
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println("Hello");
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

class WelcomeTask implements Runnable {
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println("Welcome");
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        new Thread(new MorningTask()).start();
        new Thread(new HelloTask()).start();
        new Thread(new WelcomeTask()).start();
    }
}
```

Output

```
Good Morning
Hello
Welcome
Good Morning
Good Morning
Hello
Welcome
Hello
Welcome
```

Note: *Same behaviour as the Thread version. Each class implements Runnable and is passed to a Thread.*

Exercise 7(b): isAlive() and join()

Question: Write a program illustrating isAlive() and join().

Source code: ThreadStatusDemo.java

```
public class ThreadStatusDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            System.out.println("Worker thread finished its work");
        });

        System.out.println("Before start() -> isAlive() = " + worker.isAlive());
        worker.start();
        System.out.println("After start()  -> isAlive() = " + worker.isAlive());

        worker.join();   // main waits until the worker thread finishes
        System.out.println("After join()   -> isAlive() = " + worker.isAlive());
        System.out.println("Main thread resumes");
    }
}
```

Output

```
Before start() -> isAlive() = false
After start()  -> isAlive() = true
Worker thread finished its work
After join()   -> isAlive() = false
Main thread resumes
```

Exercise 7(c): Daemon threads

Question: Write a program illustrating daemon threads.

Source code: DaemonThreadDemo.java

```
public class DaemonThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread daemon = new Thread(() -> {
            int tick = 1;
            while (true) {                      // runs forever
                System.out.println("Daemon heartbeat #" + tick++);
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    return;
                }
            }
        });

        daemon.setDaemon(true);                 // must be set before start()
        System.out.println("Is daemon thread? " + daemon.isDaemon());
        daemon.start();

        Thread.sleep(2500);                     // main thread does its work
        System.out.println("Main thread ends. JVM exits and the daemon is stopped automatically.");
    }
}
```

Output

```
Is daemon thread? true
Daemon heartbeat #1
Daemon heartbeat #2
Daemon heartbeat #3
Main thread ends. JVM exits and the daemon is stopped automatically.
```

Note: *The daemon thread loops forever, but the JVM exits as soon as the main thread finishes, so the daemon is stopped automatically. The number of heartbeats may vary slightly by timing.*

Exercise 7(d): Producer-Consumer problem

Question: Write a Java program for the Producer-Consumer problem.

Source code: ProducerConsumerDemo.java

```
class Buffer {
    private int value;
    private boolean available = false;

    public synchronized void put(int v) throws InterruptedException {
        while (available) {
            wait();                          // wait until the consumer has taken the item
        }
        value = v;
        available = true;
        System.out.println("Producer put   : " + v);
        notifyAll();
    }

    public synchronized int get() throws InterruptedException {
        while (!available) {
            wait();                          // wait until the producer has put an item
        }
        available = false;
        System.out.println("Consumer got   : " + value);
        notifyAll();
        return value;
    }
}

public class ProducerConsumerDemo {
    public static void main(String[] args) throws InterruptedException {
        Buffer buffer = new Buffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.put(i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Producer");

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.get();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Consumer");

        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
        System.out.println("Producer and consumer have finished.");
    }
}
```

Output

```
Producer put   : 1
Consumer got   : 1
Producer put   : 2
Consumer got   : 2
Producer put   : 3
Consumer got   : 3
Producer put   : 4
Consumer got   : 4
Producer put   : 5
Consumer got   : 5
Producer and consumer have finished.
```

Note: *The values are always consumed in the order 1 to 5. Only the interleaving of the 'Producer put' and 'Consumer got' lines may change between runs.*

Exercise 8: Importing and using user-defined packages

Question: Write a Java program that imports and uses user-defined packages.

Source code: mypack/MathUtils.java

```
package mypack;

// A user-defined package containing a utility class
public class MathUtils {

    public static int factorial(int n) {
        int result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    public static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static int gcd(int a, int b) {
        return (b == 0) ? a : gcd(b, a % b);
    }
}
```

Source code: PackageDemo.java

```
import mypack.MathUtils;   // import the user-defined package class

public class PackageDemo {
    public static void main(String[] args) {
        System.out.println("factorial(5)  = " + MathUtils.factorial(5));
        System.out.println("isPrime(7)    = " + MathUtils.isPrime(7));
        System.out.println("isPrime(10)   = " + MathUtils.isPrime(10));
        System.out.println("gcd(36, 48)   = " + MathUtils.gcd(36, 48));
    }
}
```

Output

```
factorial(5)  = 120
isPrime(7)    = true
isPrime(10)   = false
gcd(36, 48)   = 12
```

Note: *Files: mypack/MathUtils.java (package mypack) and PackageDemo.java (imports mypack.MathUtils). Compile with: javac -d out mypack/MathUtils.java PackageDemo.java, then run with: java -cp out PackageDemo.*

Exercise 9 (item 4): Connect to a database using JDBC

Question: Write a Java program that connects to a database using JDBC.

Source code: JdbcConnect.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

// Needs the MySQL Connector/J jar on the classpath and a running MySQL server
// with a database called "college".
public class JdbcConnect {
    static final String URL  = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            System.out.println("Connected to database successfully");
            System.out.println("Driver used : " + con.getMetaData().getDriverName());
        } catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }
}
```

Output

```
Connection failed: No suitable driver found for jdbc:mysql://localhost:3306/college
```

Note: *This program needs a running MySQL server and the MySQL Connector/J jar on the classpath. The output shown was captured in an environment without a database, so it reports a connection failure. With MySQL running on localhost and the 'college' database created, it prints the success messages.*

Exercise 9 (item b): Insert values into a database using JDBC

Question: Write a Java program to connect to a database using JDBC and insert values into it.

Source code: JdbcInsert.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

// Table assumed to exist:
//   CREATE TABLE student (id INT PRIMARY KEY, name VARCHAR(50), marks INT);
public class JdbcInsert {
    static final String URL  = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        String sql = "INSERT INTO student (id, name, marks) VALUES (?, ?, ?)";
        Object[][] rows = { {1, "Ravi", 88}, {2, "Priya", 92}, {3, "Arjun", 75} };

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = con.prepareStatement(sql)) {
            for (Object[] row : rows) {
                ps.setInt(1, (Integer) row[0]);
                ps.setString(2, (String) row[1]);
                ps.setInt(3, (Integer) row[2]);
                int count = ps.executeUpdate();
                System.out.println("Inserted " + row[1] + " -> rows affected: " + count);
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

Output

```
Database error: No suitable driver found for jdbc:mysql://localhost:3306/college
```

Note: *This program needs a running MySQL server and the MySQL Connector/J jar on the classpath. The output shown was captured in an environment without a database, so it reports a connection failure. With MySQL running on localhost and the 'college' database created, it prints the success messages.*

Exercise 9 (item c): Delete values from a database using JDBC

Question: Write a Java program to connect to a database using JDBC and delete values from it.

Source code: JdbcDelete.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class JdbcDelete {
    static final String URL  = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        String sql = "DELETE FROM student WHERE id = ?";
        int idToDelete = 2;

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, idToDelete);
            int count = ps.executeUpdate();
            System.out.println("Deleted rows with id = " + idToDelete + " -> rows affected: " + count);
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

Output

```
Database error: No suitable driver found for jdbc:mysql://localhost:3306/college
```

Note: *This program needs a running MySQL server and the MySQL Connector/J jar on the classpath. The output shown was captured in an environment without a database, so it reports a connection failure. With MySQL running on localhost and the 'college' database created, it prints the success messages.*

Exercise 8 (item 9): GUI with a label and an ImageView (JavaFX)

Question: Build a GUI that displays text in a label and an image in an ImageView (use JavaFX).

Source code: LabelImageApp.java

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class LabelImageApp extends Application {
    @Override
    public void start(Stage stage) {
        Label label = new Label("Welcome to JavaFX: Label and ImageView");

        // Put any image file (e.g., image.png) in the project folder
        Image image = new Image("file:image.png");
        ImageView imageView = new ImageView(image);
        imageView.setFitWidth(250);
        imageView.setPreserveRatio(true);

        VBox root = new VBox(15, label, imageView);
        root.setAlignment(Pos.CENTER);
        root.setStyle("-fx-padding: 20;");

        stage.setTitle("Label and ImageView Demo");
        stage.setScene(new Scene(root, 400, 380));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Note: *This is a JavaFX GUI application. JavaFX is not part of the JDK in this environment, so it was not compiled or run here. Run it in an IDE (IntelliJ, Eclipse, NetBeans) with the JavaFX SDK added to the module path. The code is the reference solution.*

Expected result: *A 400 x 380 window titled 'Label and ImageView Demo'. The label 'Welcome to JavaFX: Label and ImageView' sits above the picture from image.png, centred and scaled to 250 px wide.*

Exercise 8 (item 10): Tip Calculator app (JavaFX)

Question: Build a Tip Calculator app using several JavaFX components and learn how to respond to user interactions with the GUI.

Source code: TipCalculator.java

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.Slider;
import javafx.scene.control.TextField;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class TipCalculator extends Application {
    @Override
    public void start(Stage stage) {
        TextField billField = new TextField();
        billField.setPromptText("e.g. 250.00");

        Label tipPercentLabel = new Label("Tip: 15%");
        Slider tipSlider = new Slider(0, 30, 15);
        tipSlider.setShowTickMarks(true);
        tipSlider.setShowTickLabels(true);
        tipSlider.setMajorTickUnit(5);
        tipSlider.setBlockIncrement(1);
        tipSlider.valueProperty().addListener((obs, oldV, newV) ->
                tipPercentLabel.setText("Tip: " + newV.intValue() + "%"));

        Button calcButton = new Button("Calculate");
        Label resultLabel = new Label("Enter a bill and press Calculate");

        calcButton.setOnAction(e -> {
            try {
                double bill = Double.parseDouble(billField.getText().trim());
                double tip = bill * tipSlider.getValue() / 100.0;
                double total = bill + tip;
                resultLabel.setText(String.format("Tip: %.2f   Total: %.2f", tip, total));
            } catch (NumberFormatException ex) {
                resultLabel.setText("Please enter a valid bill amount.");
            }
        });

        GridPane grid = new GridPane();
        grid.setHgap(10);
        grid.setVgap(12);
        grid.setPadding(new Insets(20));
        grid.add(new Label("Bill amount:"), 0, 0);
        grid.add(billField, 1, 0);
        grid.add(tipPercentLabel, 0, 1);
        grid.add(tipSlider, 1, 1);
        grid.add(calcButton, 1, 2);
        grid.add(resultLabel, 0, 3, 2, 1);

        stage.setTitle("Tip Calculator");
        stage.setScene(new Scene(grid, 460, 280));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Note: *This is a JavaFX GUI application. JavaFX is not part of the JDK in this environment, so it was not compiled or run here. Run it in an IDE (IntelliJ, Eclipse, NetBeans) with the JavaFX SDK added to the module path. The code is the reference solution.*

Expected result: *A bill text field, a tip slider (0 to 30 percent, default 15) with a live label, and a Calculate button. Entering 250 with 15 percent and pressing Calculate shows 'Tip: 37.50 Total: 287.50'. A non-number shows 'Please enter a valid bill amount.'*